

NAME: SAMRUDDHI SUNIL BATE

ROLL NO: 14

PRN NO: 12520064

BATCH: 01

CLASS: SY-CS-SEDA-B

SUBJECT: OS

Operating System Simulation (Phase-2)

○ Aim:

To simulate an Operating System with paging, interrupts, and error handling mechanisms.

○ Objectives:

No. Objective

1	To understand paging and memory management
2	To implement address mapping (Virtual → Real)
3	To handle interrupts (SI, PI, TI)
4	To implement error handling
5	To control program execution using TTL and TLL

○ Theory :

Phase-2 is an advanced version of Phase-1 with additional features like paging and interrupt handling.

Key Concepts:

- **Paging:** Memory is divided into pages and frames
- **Page Table:** Stores mapping of pages to frames
- **PTR (Page Table Register):** Points to page table location
- **Address Mapping:** Converts Virtual Address (VA) to Real Address (RA)

○ Registers Used :

Register Description

IC	Instruction Counter
IR	Instruction Register
PTR	Page Table Register
TTC	Total Time Counter
LLC	Line Limit Counter

○ Interrupts :

Interrupt Meaning

SI (Service Interrupt)	GD, PD, H operations
PI (Program Interrupt)	Opcode/Operand/Page Fault
TI (Time Interrupt)	Time limit exceeded

○ Error Codes :

Code Meaning

0	No Error (Normal Execution)
1	Out of Data
2	Line Limit Exceeded
3	Time Limit Exceeded
4	Operation Code Error

5	Operand Error
6	Invalid Page Fault

○Instructions Used :

Instruction Function

GD	Read data from input
PD	Print data to output
H	Stop execution



○Algorithm

Step Description

1	Start the program
2	Initialize memory and page table
3	Read \$AMJ → extract TTL and TLL
4	Load program into memory using paging
5	On \$DTA → start execution
6	Map Virtual Address to Real Address
7	Fetch instruction into IR
8	Execute instruction (GD, PD, H)
9	Handle interrupts using MOS
10	Check errors and limits
11	Stop when job ends

○Program :

```
#include<iostream>
#include<fstream>
#include<cstring>
#include<cstdlib>
#include<cctype>
using namespace std;
```



```
char M[300][4], IR[4];
int IC, SI, PI, TI;
int PTR;

int TTL, TLL, TTC, LLC;
bool endJob=false;

ifstream fin("input.txt");
ofstream fout("output.txt");

char buffer[40];

// INIT
void init()
{
    for(int i=0;i<300;i++)
        for(int j=0;j<4;j++)
            M[i][j]=' ';

    IC=0; SI=0; PI=0; TI=0;
    TTC=0; LLC=0;
    endJob=false;

    PTR=200;

    for(int i=0;i<10;i++)
        for(int j=0;j<4;j++)
            M[PTR+i][j]='*';
}
```

```

// ADDRESS MAP
int mapAddress(int VA)
{
    if(VA>=100)
    {
        PI=2;
        return -1;
    }

    int page = VA/10;
    int offset = VA%10;

    if(M[PTR+page][0]=='*')
    {
        PI=3;
        return -1;
    }

    int frame = (M[PTR+page][2]-'0')*10 + (M[PTR+page][3]-'0');

    return frame*10 + offset;
}

// TERMINATE
void terminate(int err)
{
    fout<<"\nJob End. Error Code: "<<err<<endl;
    endJob=true;
}

```

```

// MOS
void MOS()
{
    // VALID PAGE FAULT (GD)
    if(PI==3 && IR[0]=='G')
    {
        int page=(IR[2]-'0')*10 + (IR[3]-'0');
        page/=10;

        int frame = rand()%20;

        M[PTR+page][0]='0';
        M[PTR+page][1]='0';
        M[PTR+page][2]=(frame/10)+'0';
        M[PTR+page][3]=(frame%10)+'0';

        PI=0;
        return;
    }

    // ERRORS
    if(PI==1){ terminate(4); return; }
    if(PI==2){ terminate(5); return; }
    if(PI==3){ terminate(6); return; }
    if(TI==2){ terminate(3); return; }

    // GD
    if(SI==1)
    {
        if(!fin.getline(buffer,40) || buffer[0]=='$')

```

```

{
    terminate(1);
    return;
}

int RA = mapAddress((IR[2]-'0')*10 + (IR[3]-'0'));
if(PI!=0){ MOS(); return; }

int k=0;
for(int i=0;i<10;i++)
{
    for(int j=0;j<4;j++)
    {
        if(buffer[k]=='\0') break;
        M[RA][j]=buffer[k++];
    }
    RA++;
}

}

// PD
else if(SI==2)
{
    if(LLC >= TLL)
    {
        terminate(2);
        return;
    }

    int RA = mapAddress((IR[2]-'0')*10 + (IR[3]-'0'));

```

```
if(PI!=0){ MOS(); return; }
```

```
for(int i=0;i<10;i++)
```

```
{
```

```
    for(int j=0;j<4;j++)
```

```
        if(M[RA][j]!=' ')
```

```
            fout<<M[RA][j];
```

```
        RA++;
```

```
    }
```

```
    fout<<endl;
```

```
    LLC++;
```

```
}
```

```
// H
```

```
else if(SI==3)
```

```
{
```

```
    terminate(0);
```

```
}
```

```
}
```

```
// EXECUTE
```

```
void execute()
```

```
{
```

```
    while(true)
```

```
    {
```

```
        if(endJob) break;
```

```
        int RA = mapAddress(IC);
```

```
        if(PI!=0)
```

```
{
    MOS();
    continue;
}

for(int i=0;i<4;i++)
    IR[i]=M[RA][i];

IC++;
TTC++;

if(TTC > TTL)
{
    TI=2;
    MOS();
    break;
}

// operand check
if(IR[0]!='H' && !(isdigit(IR[2]) && isdigit(IR[3])))
{
    PI=2;
    MOS();
    break;
}

// instructions
if(IR[0]=='G' && IR[1]=='D')
{
    SI=1; MOS();
```

```

    }
    else if(IR[0]=='P' && IR[1]=='D')
    {
        SI=2; MOS();
    }
    else if(IR[0]=='H')
    {
        SI=3; MOS();
    }
    else
    {
        PI=1; MOS();
    }

    if(endJob) break;

    SI=0;
}
}

// LOAD
void load()
{
    int m=0, frame=0;
    bool program=true;

    while(fin.getline(buffer,40))
    {
        if(strncmp(buffer,"$AMJ",4)==0)
        {

```

```
init();

// READ TTL & TLL
char temp[5];

strncpy(temp, buffer+8, 4);
temp[4]='\0';
TTL = atoi(temp);

strncpy(temp, buffer+12, 4);
temp[4]='\0';
TLL = atoi(temp);

m=0;
frame=0;
program=true;
}

else if(strncmp(buffer,"$DTA",4)==0)
{
    program=false;
    execute();
}

else if(strncmp(buffer,"$END",4)==0)
{
    continue;
}

else if(program)
```

```

{
    int page = m/10;

    if(M[PTR+page][0]!='*')
    {
        M[PTR+page][0]='0';
        M[PTR+page][1]='0';
        M[PTR+page][2]=(frame/10)+'0';
        M[PTR+page][3]=(frame%10)+'0';
        frame++;
    }

    int f = (M[PTR+page][2]-'0')*10 + (M[PTR+page][3]-'0');
    int RA = f*10 + (m%10);

    int k=0;
    for(int j=0;j<4;j++)
        M[RA][j]=(buffer[k]!='\0')?buffer[k++]:' ';

    m++;
}
}

// MAIN
int main()
{
    load();
    fin.close();
    fout.close();
}

```

```
cout<<"Phase-2 Done (All Errors Working)"<<endl;  
}
```

● Input :

\$AMJ000300050005

GD10

PD10

H

\$DTA

HELLO

\$END

\$AMJ000300050005

GD10

GD20 H

\$DTA

HELLO

\$END

\$AMJ000300050005

PD10

H

\$DTA

HI

\$END

\$AMJ000300010003

GD10

GD20

GD30 H

\$DTA

HI

HI

HI

\$END

\$AMJ000100030001

GD10 PD10

PD10

H

\$DTA

HI

\$END

\$AMJ000300050005

XX10 H

\$DTA

HELLO

\$END

\$AMJ000300050005

GD1A

H

\$DTA

HELLO

\$END

OUTPUT:

Job End. Error Code: 0

Job End. Error Code: 1

Job End. Error Code: 6

Job End. Error Code: 3
GD10PD10PD10H

Job End. Error Code: 2

Job End. Error Code: 4

Job End. Error Code: 5

○Output Explanation :

Case	Error Code	Reason
Normal	0	Program executed successfully
Out of Data	1	No data for GD instruction
Page Fault	6	Invalid page access
Time Limit	3	TTC exceeded TTL
Line Limit	2	LLC exceeded TLL
Opcode Error	4	Invalid instruction
Operand Error	5	Invalid operand

○Advantages :

No.	Advantage
1	Supports paging
2	Handles errors
3	Uses interrupts
4	Better memory management

○Limitations:

No.	Limitation
1	No multiprogramming
2	Simple paging only
3	Limited instruction set
4	No protection mechanisms

○Conclusion :

Phase-2 successfully simulates an advanced Operating System with paging, interrupts, and error handling. It improves upon Phase-1 by introducing memory management and system control mechanisms.